Spherical to cartesian

In[∘]:=
```
CreateVectors[inputArray_] :=
 Module[{theta1, theta2, deltaPhi, L, S1, S2, LVector, S1Vector, S2Vector, phi1, phi2},
 módulo
  {theta1, theta2, deltaPhi} = inputArray;
  L = 7 / 10;
  S1 = 3 / 10;
  S2 = 4 / 10;
  phi1 = 0;
  phi2 = phi1 - deltaPhi;
  LVector = {0, 0, L};
  S1Vector = {S1 Sin[theta1] Cos[phi1], S1 Sin[theta1] Sin[phi1], S1 Cos[theta1]};
                 seno        coseno          seno        seno          coseno
  S2Vector = {S2 Sin[theta2] Cos[phi2], S2 Sin[theta2] Sin[phi2], S2 Cos[theta2]};
                 seno        coseno          seno        seno          coseno
  {LVector, S1Vector, S2Vector}]

AngleVectors = {θ1, θ2, Δϕ} /.    {θ1 →
     2.176803881671145622971774945168283469575682886385507757747021954544455114076 33`
      50.,
    θ2 →
     1.994789770458220505535613855872614179758079800532807200477621705202970630028 61`
      50.,
    Δϕ →
     1.384089475898462340082399818216702082249068066417655473837693604301204348845 38`
      50.}    ;
VectorVector = CreateVectors[AngleVectors]
```

Out[∘]=

$$\left\{\left\{0, 0, \frac{7}{10}\right\}, \{0.246578586371277169157638117910134901813275520149 96,\right.$$
$$0, -0.170877151027112472431939543142761113348670754681 62\},$$
$$\{0.067675039101556281823884408833499939716812394246 25,$$
$$-0.358245204648429723462124826425979476677346359557 37,$$
$$\left.-0.164561424486443768691622250493391073094111113228 79\}\right\}$$

Rotation vector to keep spins constant

In[∘]:=
```
RotateVectorAroundAxis[L_, S_, phi_] :=
  Module[{SUnit, LParallel, LPerpendicular, RotationMatrix, LFinal},
  módulo                                    matriz de rotación
   SUnit = Normalize[S];
           normaliza
   LParallel = (L.SUnit) SUnit;
   LPerpendicular = L - LParallel;
   RotationMatrix = RotationTransform[phi, SUnit];
   matriz de rotación  transformación de rotación
   LFinal = LParallel + RotationMatrix[LPerpendicular];
                        matriz de rotación
   LFinal];
```

System equations

In[ ]:=
```
(*m2 < m1 in Gerosa. q of Gerosa is 1/q of Racine.*)
```

In[ ]:=
```
m1 = 1;
m2 = 1 / 2 ;
S1init = {-2, 5, 3} ;
S2init = 2 S1init + -2 {2, 1, 0};
Linit0 = S2init + {1, 3, 1} ;
Sinit = S1init + S2init;
phiRotationAngle = 1 ;
Linit = RotateVectorAroundAxis[Linit0, Sinit, phiRotationAngle];

{Linit, S1init, S2init} = VectorVector ;

end = 80;
d = 1;

M = m1 + m2;
```

$$\mu = \frac{m1\, m2}{M};$$

$$q = \frac{m1}{m2};$$

$$S0 = \left(1 + \frac{m2}{m1}\right) S1[t] + \left(1 + \frac{m1}{m2}\right) S2[t];$$

```
S = S1[t] + S2[t];
```

$$\lambda = \frac{\left(1 + \frac{m2}{m1}\right) L[t].S1[t] + \left(1 + \frac{m1}{m2}\right) L[t].S2[t]}{\text{Norm}[L[t]]^2};$$

$$\text{sol1} = \text{NDSolve}\Big[\Big\{S1'[t] == \frac{1}{2\, d^3}\, \text{Cross}\Big[\Big(4 + 3\frac{m2}{m1} - 3\frac{\mu}{m1}\lambda\Big) L[t] + S2[t],\ S1[t]\Big],$$

resolvedor diferencial numérico producto vectorial

$$S2'[t] == \frac{1}{2\, d^3}\, \text{Cross}\Big[\Big(4 + 3\frac{m1}{m2} - 3\frac{\mu}{m2}\lambda\Big) L[t] + S1[t],\ S2[t]\Big],$$

producto vectorial

$$L'[t] == \frac{1}{2\, d^3}\, \text{Cross}\Big[S + 3\Big(1 - \frac{\mu}{M}\lambda\Big) S0,\ L[t]\Big]$$

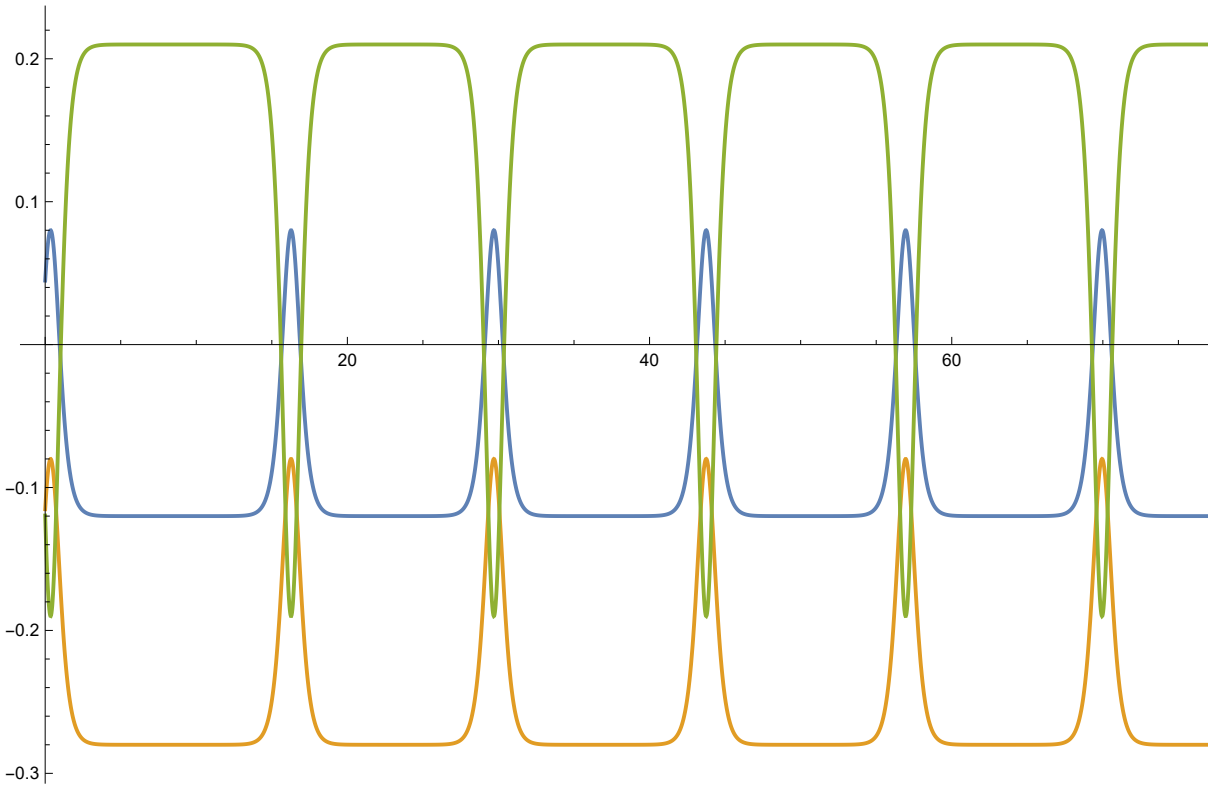
producto vectorial

$$,\ S1[0] == S1init,\ S2[0] == S2init,\ L[0] == Linit\Big\},\ \{S1, S2, L\}, \{t, 0, end\}\Big];$$

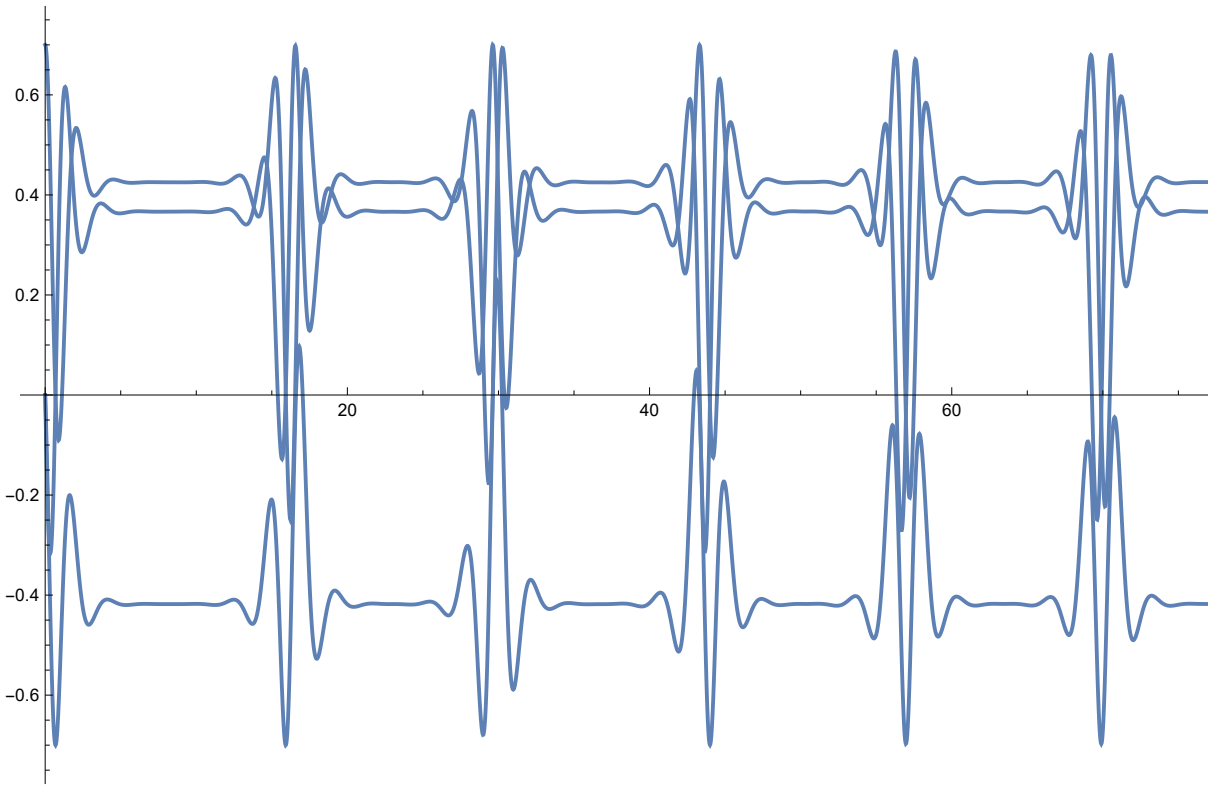
In[ ]:=
```
S1 = (S1 /. sol1[[1]]) ;
S2 = (S2 /. sol1[[1]]) ;
L = (L /. sol1[[1]]) ;


Plot[{S1[t].S2[t], L[t].S2[t], S1[t].L[t]} , {t, 0, end}]
```
representación gráfica

```
Plot[{L[t]} , {t, 0, end}]
```
representación gráfica

Out[•]=



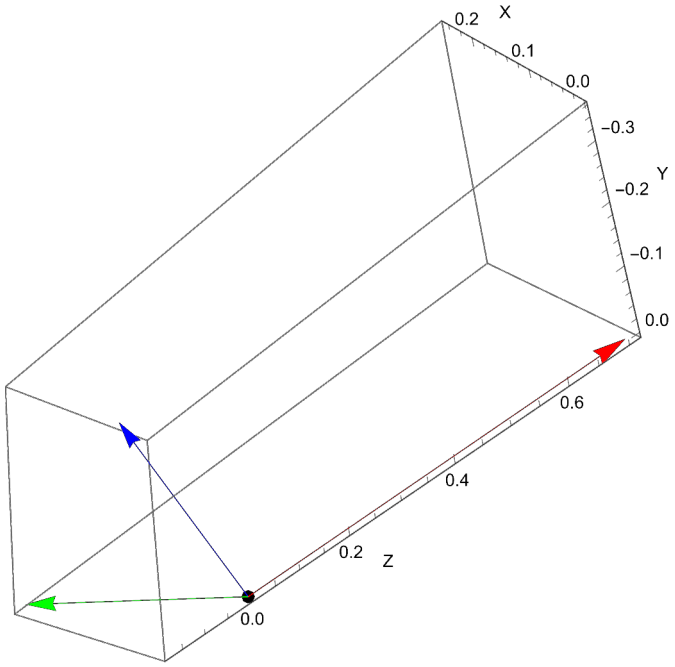
Out[•]=



Visualize vectors in 3D space

In[ ]:=
```
VisualizeVectors3D[vectors_List] := Module[{arrows, colors},
                   lista                 módulo
  colors = {Red, Green, Blue};
           rojo  verde  azul

  arrows = MapThread[{#1, Arrow[{{0, 0, 0}, #2}]} &, {colors, vectors}];
           aplica a través  flecha
  Graphics3D[{arrows, {Black, PointSize[Large], Point[{0, 0, 0}]} },
  gráfico 3D          negro  tamaño de···  grande  punto
   Axes → True, AxesLabel → {"X", "Y", "Z"}, Boxed → True, ViewPoint → {2, 2, 2}]]
   ejes  verd···  etiqueta de ejes            rodead···  verd···  punto de vista
VisualizeVectors3D[VectorVector]
```

Out[ ]=



In[ ]:=
```
Quit[];
detén núcl
```

In[ ]:=
```
10 000  // Pause;
           pausa
```

In[∘]:=
```
Quit[];
```
detén núcl

Angle test

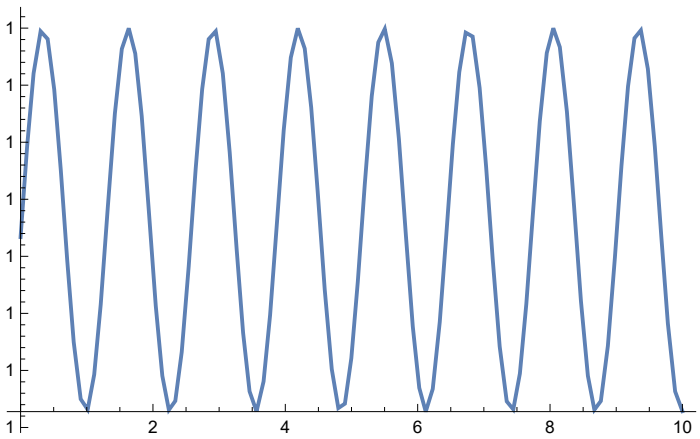
```
J = L[t] + S1[t] + S2[t];
zcap = J / Norm[J] ;
xcap = (L[t] - zcap.L[t] zcap) / Norm[L[t] - zcap.L[t] zcap];
ycap = Cross[zcap, xcap];
```
producto vectorial

In[∘]:=
```
Sperp = Cross[ycap, S1[t] + S2[t]] ;
```
producto vectorial

```
SperpNorm = Sperp / Norm[Sperp] ;
S1norm = S1[t] / Norm[S1[t]] ;
Snorm = S / Norm[S] ;
```

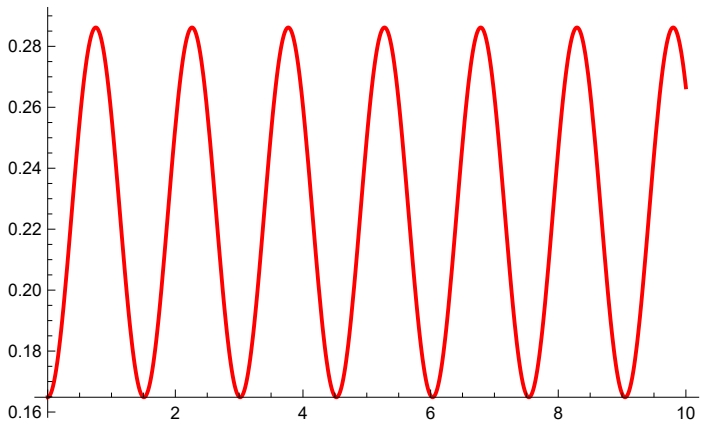
In[∘]:=
```
Plot[ S1norm.SperpNorm / Norm[Cross[S1norm, Snorm]] , {t, 0, end}]
```
representación gráfica

Out[∘]=



In[•]:=
```
Plot[ (S1[t])[[3]] , {t, 0, end}, PlotStyle → Red]
```
representación gráfica estilo de repr··· rojo

Out[•]=



In[•]:=
```
AllThreePhasesPossible = (Linit + S1init + S2init // Norm ) >
    (S1init // Norm) + (S2init // Norm) - (Linit // Norm);
```
norma norma norma norma norma

```
ξTestFunc[phiRotationAngle_] := M^-2 ((1 + q^-1) S1init + (1 + q) S2init).
    (RotateVectorAroundAxis[Linit0, Sinit, phiRotationAngle]) /
   Norm[RotateVectorAroundAxis[Linit0, Sinit, phiRotationAngle]]
```
norma

```
Smin = Max[Abs[(Linit + S1init + S2init // Norm ) - ( Linit // Norm)],
    Abs[(S1init // Norm ) - ( S2init // Norm)] ] // N ;
```
má·· valor absoluto norma norma valor absoluto norma norma valor numérico

```
Smax = Min[(Linit + S1init + S2init // Norm ) + ( Linit // Norm) // N,
    (S1init // Norm ) + ( S2init // Norm) // N] ;
```
mínimo norma norma val norma norma valor numérico

In[•]:=
```
Clear[J, L, S, S1, S2];
```
borra

```
A1 = (J^2 - (L - S)^2)^(1/2) ;
A2 = (-J^2 + (L + S)^2)^(1/2) ;
A3 = (S^2 - (S1 - S2)^2)^(1/2) ;
A4 = ((S1 + S2)^2 - S^2)^(1/2) ;
ξPlus =
  ((J^2 - L^2 - S^2) (S^2 (1 + q^-1)^2 - (S1^2 - S2^2) (1 - q^-2)) + (1 - q^-2) A1 A2 A3 A4) / (4 q^-1 M^2 S^2 L) ;

S1 = S1init;
S2 = S2init;
L = Linit ;
J = S1init + S2init + Linit;
S1 = S1 // Norm;
          norma
S2 = S2 // Norm;
          norma
L = L // Norm;
          norma
J = J // Norm;
          norma

ξ1 = ξPlus /. S → Smin // Re
                          parte real
ξ2 = ξPlus /. S → Smax // Re
                          parte real
ξTest = ξTestFunc[phiRotationAngle]
Plot[{ξTestFunc[x], ξ1(*,ξ2*)} , {x, 2, 4}]
representación gráfica
FindRoot[ξTestFunc[x] - ξ1 , {x, 0, 0, 4}]
encuentra raíz
(*This root-finding finds the rotation angle which lands us on the "separatrix"*)
```

Out[•]=

0.441004

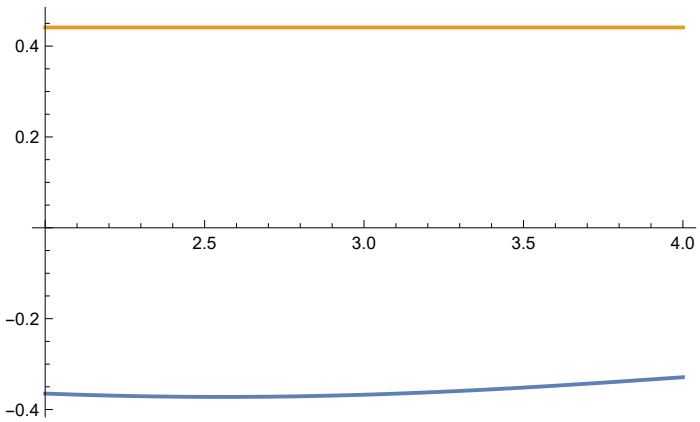
Out[•]=

0.138714

Out[•]=

-0.356624

Out[•]=



FindRoot: The point {0.} is at the edge of the search region {0., 4.} in coordinate 1 and the computed search direction points outside the region.

Out[•]=

```
{x → 0.}
```

In[•]:=

```
AllThreePhasesPossible
If[AllThreePhasesPossible,
 If[ξTest > Max[ξ1, ξ2],
  Decision = AroundPi];
 If[ξTest < Min[ξ1, ξ2],
  Decision = Around0];
 If[ξTest > Min[ξ1, ξ2] && ξTest < Max[ξ1, ξ2],
  Decision = AllTheWay];
]
Decision
```

Out[•]=

```
True
```

Out[•]=

```
Around0
```

In[•]:=

```
Quit[];
```